

КЕНГА

Язык, который компилирует
и учит себя

Герман Янтарас

при участии kenga-lit 3.13

КЕНГА

Язык, который компилирует и учит себя

*От hello до модели, которая пишет программу:
диалект, хост без Rust, компилятор на Kenga
и честный масштаб своей LM*

Герман Янтарас

при участии kenga-lite 3.13

Первое издание · 2026

© Герман Янтарас, 2026

Эта книга описывает язык Kenga таким, каким он есть в репозитории `kenga-lang` на момент первого издания: версия хоста 3.13, канон без Rust в каталогах `bootstrap/` и `kenga/`. Все команды воспроизводимы. Если эксперимент не повторяется — виноват текст, не «дух языка».

Книга написана человеком. Соавтор по коду — тот же рабочий контур, что собирает `kenga-lite`. Это не рецензия индустрии и не обещание модели уровня Grok.

Оглавление

Введение. Язык, который не врёт про себя	6
Часть I. Язык	7
1. Что такое Kenga	8
2. Установка без Rust	9
3. Диалект	10
4. События и агент	11
Часть II. Свобода	12
5. Почему не Rust	13
6. Каркас kenga-lite	14
7. Карта kenga/	15
Часть III. Компилятор на себе	16
8. more.kenga	17
9. Петля birth	18
10. Kenga пишет C	19
Часть IV. Память и тензор	20
11. Prophet Memory	21
12. Тензор и лента	22
13. XOR на list/f64	23
Часть V. Своя модель	24
14. Decoder, не скачанный граф	25
15. Подписи, которые модель умеет	26
16. Честный масштаб	27
Часть VI. Практикум	28
17. Задачи	29

18. Лабораторный журнал	31
19. Куда смотреть в репозитории	32
Послесловие	33

Введение. Язык, который не врёт про себя

Kenga — язык программирования для живого ИИ: память, предсказание, агентный цикл и тензор живут в семантике, а не в обёртке над чужим фреймворком. Это коротко. Дальше начинается то, ради чего стоит держать книгу рядом с репозиторием.

Обычный путь «языка для ИИ» выглядит так: синтаксис поверх Python, веса скачаны с Hugging Face, доказательство — чужой GGUF в папке с вашим логотипом. Мы этот путь откладываем. Доказательство Kenga — сеть, написанная на Kenga и обученная на Kenga. Пока сеть крошечная. Это не стыдно, если не называть её половиной чужой модели.

Вторая нить книги — свобода от Rust. Хост kenga-lite собирается C99-компилятором. Файл bootstrap/kenga_lite.c почти пустой: CRT и цепочка `#include "generated/rt_*.inc.c"`. Сами `rt_*` пишет Kenga. Компилятор `more.kenga` уже запускает чужие программы на своём VM, в том числе ту, которую написала suffix-модель: `fact(add(2,3)-1)` печатает 24.

«Скачанный GGUF не доказывает язык. Доказывает архитектура, обучение и корпус в .kenga.»

Структура. Часть I — диалект, чтобы открыть файл и не утонуть. Часть II — зачем уходить с Rust и как устроен каркас. Часть III — компилятор на Kenga и петля `birth`. Часть IV — память, тензор, XOR. Часть V — своя языковая модель и честная таблица того, чего нет. Часть VI — практикум и лабораторный журнал. Приложение — карта файлов и встроенные функции.

Тон тот же, что в книге по Z-системе: аксиома без эксперимента — декор. Отрицательный результат оставляем в тексте.

ЧАСТЬ I

Язык

Глава 1

Что такое Kenga

Kenga — императивный язык с функциями, списками, структурами, событиями и встроенным тензором. Хост 3.13 умеет два пути: полный бинарник с Releases (ещё содержит Rust) и kenga-lite из bootstrap/, который Rust не вызывает.

F1. Живой runtime без Rust существует: Prophet Memory, тензор, tape, события, компилятор и VM собираются из эмиттеров в kenga/emit/.

F2. Расширение VS Code на маркетплейсе — 3.13.0. Язык уже дальше (word-decoder, rt_* хост). Номер расширения не двигаем, пока не меняется сам редактор.

Чем язык не является

Не ChatGPT из коробки. Не замена PyTorch. Не pretrained CLIP или Whisper. Не Grok и не чужой GGUF под другим именем. Сид для Hugging Face лежит в hf/kenga-seed/; большую карточку Kenga-ai/kenga-mm туда класть рано.

Да	Нет
Свой язык, VM, living memory	Готовый чат-бот
Decoder в .kenga, крошечный корпус	Словарь 50k и контекст 4k
birth → программа → 24	Доказательство через GGUF
Kenga пишет C99 (c_seed, lower_c)	VM без C-хоста

Если знакомый спрашивает «это как Python для нейросетей?» — нет. Python клеит C++/CUDA. Kenga держит тензор и агентный цикл как часть языка. Масштаб пока учебный. Это разные утверждения, и оба верны одновременно.

Глава 2

Установка без Rust

Друзьям проще начать с бинарника. Клонировешь репозиторий — нужны examples/ — кладёшь кенга в PATH, запускаешь demo.

Сборка lite

```
git clone https://github.com/GermannM3/kenga-lang.git
cd kenga-lang
bootstrap\build.cmd
bootstrap\bin\kenga-lite.exe version
```

build.cmd гоняет selftest: 52 случая. Если падает — чини хост, не книгу. Дымовой контур свободы: scripts/freedom-smoke.cmd.

Полный cargo install остаётся для GPU и legacy-пути. Новый код в src/ не кладём. Карта замены: docs/REPLACE_RUST.md. Путь к самостоятельности: docs/INDEPENDENCE.md.

Первый запуск

examples/hello.kenga

```
fn main() -> i64 {
    println("hello from kenga");
    let x: i64 = 21;
    let y: i64 = 2;
    println(x * y);
    return 0;
}
```

Запуск: kenga run --lite examples/hello.kenga — или тот же файл через kenga-lite.exe run. Печатает приветствие и 42.

Дальше по вечеру: docs/LEARN.md. Справочник синтаксиса: docs/LANGUAGE.md. Упражнения без подсказок из examples/: docs/EXERCISES.md.

Глава 3

Диалект

Lite-диалект уже покрывает то, без чего не напишешь компилятор и маленькую сеть: целые, f64, строки, списки, struct, if / else if, while, for, break, continue, вызовы функций вперёд по тексту.

Условия и циклы

if cond { ... } else if ... else { ... }. while cond { ... }. for i in 0..n и for x in xs. Смотри [examples/showcase.kenga](#) и [examples/selfhost/for_lite.kenga](#).

Списки и структуры

список и struct

```
let xs = [1, 2, 3];
xs = push(xs, 4);
println(len(xs));

struct Point { x, y }
let p = Point { x: 3, y: 4 };
println(p.x);
```

Шпаргалки: [examples/selfhost/lists_lite.kenga](#), [struct_lite.kenga](#). [lower_c.kenga](#) уже умеет опустить struct в C.

Ограничения, о которые бьёшься руками

В lite нет && и || — пиши вложенный if. Подчёркивание в именах можно. Не называй свою функцию predict: это встроенная. Не гони strip_line_comments по огромному файлу перед take — intern там квадратичный, зависает незаметно.

Строки в more.kenga долго ломались на экранах. Старый parse_string считал \" концом строки. Из-за этого c_seed.kenga раздувался до сотен мегабайт и его убивали. Когда экраны починили, more стал прогонять c_seed и expr_c и писать .c. Это не «красивый рефакторинг». Это баг, без которого петля emit не закрывается.

Глава 4

События и агент

Агент в Kenga — не фреймворк. Это `on` / `emit` / `pump` / `pending`. Обработчик подписывается на имя события, `emit` кладёт сообщение в очередь, `pump` её разбирает.

схема событий

```
on "tick"(n) {  
    println(n);  
}  
emit("tick", 1);  
pump();
```

Живой пример: `examples/agent.kenga`. На `lite`: `kenga run --lite examples\agent.kenga`. Тот же цикл уже есть в `more.kenga` — компилятор на Kenga гоняет агента без отдельного C-VM.

Chat-интенты (`kenga chat --lite minds/agent.km`) сидят в `kenga/emit/rt_chat.kenga`. Это не языковая модель. Это разбор фраз в духе «смотри 5 1 6» и «что будет завтра?» поверх Prophet.

ЧАСТЬ II

Свобода

Глава 5

Почему не Rust

Rust-хост в `src/` умеет больше и держит GPU-путь. Он же якорь: пока новый смысл пишется на Rust, язык не живёт на себе. Правило репозитория простое. Новый код — в `kenga/` или `bootstrap/`. `src/` не расширяем. Когда Releases станут `lite-only`, `src/` уйдёт в архив.

F3. C и Rust — подмости. Цель — VM на Kenga, потом убрать C.

Слой	Без Rust	Без C
Living runtime (<code>rt_*</code>)	да	нет, gcc/cl
<code>more.kenga</code> : <code>birth</code> , <code>XOR</code> , <code>c_seed</code>	да	нет
<code>lower_c</code> / <code>lower_kv</code>	да	нет, линкер
Полный CLI <code>src/</code>	ещё legacy	—

Лестница из `docs/INDEPENDENCE.md`. Сначала `lite`-хост. Потом `kenga/compiler` вытесняет `compiler.rs` и кусок VM. Потом `lower_*` вытесняет `codegen.rs`. Потом `bc_src_c`: разбор `.kenga` в байткод и сгенерированную C-VM. Дальше — VM на Kenga без C-хоста. Releases без cargo.

«kenga_lite.c должен оставаться каркасом includes. Новую логику хоста руками туда не пишем — эмитим из .kenga.»

Глава 6

Каркас kenga-lite

После эмиттеров `bootstrap/kenga_lite.c` — около девяноста строк: заголовки `CRT` и `#include generated/rt_*.inc.c`. Компилятор, `VM`, `selftest`, `Prophet`, тензор, `tape`, события, `chat`, типы и опкоды приходят из `Kenga`.

Эмиттеры лежат в `kenga/emit/`. Имена честные: `rt_lex`, `rt_parse`, `rt_expr`, `rt_factor`, `rt_stmt`, `rt_compile`, `rt_vm`, `rt_selftest`, `rt_prophet`, `rt_tensor`, `rt_tape`, `rt_events`, `rt_chat`. Пересобрал эмиттер — пересобрал `host`.

три команды каркаса

```
bootstrap\build.cmd
scripts\freedom-smoke.cmd
bootstrap\bin\kenga-lite.exe run kenga\compiler\more.kenga
```

Старые ручные файлы `bootstrap/prophet_lite.inc.c` и соседи удалены, когда соответствующие `rt_*` стали источником. Если поправишь `host` и тянешься к `.c` вместо `.kenga` — ты идёшь назад по лестнице.

Глава 7

Карта kenga/

Каталог kenga/ — каноническая замена src/*.rs. Не зеркало и не прокладка. Сюда кладём то, что должно остаться, когда Rust уйдёт.

Путь	Роль
compiler/lite.kenga	узкий compiler+VM на i64
compiler/more.kenga	birth→24, XOR, c_seed/expr_c
emit/c_seed.kenga	Kenga пишет .c
emit/expr_c.kenga	выражение → C99 + проверка
emit/lower_c.kenga	while/if/for/struct/f64 → C
emit/rt_*.kenga	куски runtime → generated/*.inc.c

Полная таблица — в kenga/README.md и docs/REPLACE_RUST.md. Эта глава не дублирует каждую строку. Смысл один: если модуль ещё живёт только в Rust, у него должно быть имя файла в kenga/, куда он переедет.

ЧАСТЬ III

Компилятор на себе

Глава 8

more.kenga

more.kenga — компилятор и VM, написанные на Kenga. Он читает исходник, собирает байткод, исполняет. На нём уже ходят `fact_lite`, `kenga_net.kenga` (XOR), `kenga_birth.kenga` и `kenga_born.kenga`.

F4. more запускает `examples/ml/kenga_born.kenga` и печатает 24. Отдельный C-VM для этой проверки не нужен.

Опкоды `read_file` / `write_file` — 32 и 33. Без них `birth` не может положить программу на диск, а `c_seed` не может выписать `.c`.

Экраны в `parse_string`: `\n`, `\t`, `\"`, `\\`. Пока их не было, `emit`-файлы с кавычками внутри строк вешали парсер. После правки `more` прогоняет `kenga/emit/c_seed.kenga` и `expr_c.kenga`.

Что more ещё не умеет

Тензорные опкоды `decoder`'а на VM `more` пока не живут. Триграмма на `more` может быть слишком медленной, чтобы считать её повседневным тестом. Это дыры, не «скоро». Пока `decoder` гоняет `kenga-lite`, не `more`.

Глава 9

Петля birth

kenga_birth.kenga — suffix language model. Она дописывает seed с fn add, кладёт examples/ml/kenga_born.kenga, и уже другой запуск исполняет написанное. Ожидаемый вывод — 24: факториал от add(2,3)-1.

то, что пишет модель (сокращено)

```
fn add(a: i64, b: i64) -> i64 {
  return a + b;
}
fn fact(n: i64) -> i64 {
  let p = 1;
  let i = 1;
  while i <= n {
    p = p * i;
    i = i + 1;
  }
  return p;
}
fn main() -> i64 {
  let x = add(2, 3);
  let y = sub(x, 1);
  if y > 0 { println(fact(y)); } else { println(0); }
  return 0;
}
```

Команды. scripts\kenga-birth.cmd — привычный контур. scripts\bc-run.cmd examples\ml\kenga_birth.kenga — birth через native C без lite VM, потом kenga-lite run examples\ml\kenga_born.kenga. Или сразу more: он сам пишет born и исполняет.

F5. Birth — не GGUF. Это suffix-LM на корпусе языка. Программа, которую она пишет, короткая и заранее известная по форме. Ценность не в «творчестве», а в замкнутом круге: модель на Kenga производит Kenga, хост на Kenga это исполняет.

«24 — не магия. Это fact(4). Магия была бы, если бы мы выдали 24 и спрятали, кто написал файл.»

Глава 10

Кенга пишет C

`c_seed.kenga` и `expr_c.kenga` — зародыш кодогенерации. Язык печатает C99, его можно скомпилировать снаружи. `lower_c` идёт дальше: `while`, `if`, `for`, списки, `struct`, `else if`, `f64`, `import`.

`rt_kval` + `lower_kv` — `tagged runtime`: строки, `hetero`-списки, события, файловый ввод-вывод. `bc_src_c` разбирает исходник в байткод и выплёвывает C-VM. На этой сетке уже сидят `kenga_net` и `kenga_birth`.

Это всё ещё C на выходе. Свобода здесь в авторе: исходник генератора — `.kenga`, не `codegen.rs`. Когда VM more покроем тот же контур без `gcc`, ступень можно вычеркнуть.

ЧАСТЬ IV

Память и тензор

Глава 11

Prophet Memory

Prophet — живая память агента: memory, learn, predict, unroll, surprise, consolidate. Сохранение ума — save_mind / load_mind, формат .km совместим с Rust-путём. Реализация без Rust: kenga/emit/rt_prophet.kenga.

Учебные файлы: examples/ml/world_model.kenga, surprise_gate.kenga. Диалог: kenga chat --lite minds/agent.km. Это world-model на маленьких рядах, не языковая модель из части V.

ttl и консолидация входят в семантику языка, потому что агент без забывания — это лог, а не память. В Z-системе коллапс спектра — физика забвения весов. Здесь ttl — бытовой рычаг той же мысли: не всё стоит помнить вечно.

Глава 12

Тензор и лента

Плотный f64-тензор: `t_from`, `matmul`, поэлементные операции, `reshape`, `softmax`. Источник: `kenga/emit/rt_tensor.kenga`. Проверка: `examples/ml/tensor_core.kenga`.

Обратный проход — `tape`, `rt_tape.kenga`. Примеры: `autograd_tape.kenga`, `mlp_autograd.kenga`. SGD: `train_sgd.kenga`, `t_sgd_step`. Картинка: `load_ppm`, `t_mean`. Звук: `load_wav`. Fusion кадра и тона — в `multimodal`-примерах, не в отдельном «CLIP».

единичная матрица

```
let I = t_from([2, 2], [1, 0, 0, 1]);
let v = t_from([2, 1], [3, 4]);
let y = t_matmul(I, v);
# ожидай [3, 4]
```

Vis-bias у word-decoder сидит на логитах (`whead @ fuse`), не в магии эмбединга. `whead` учим только на токене цвета, не на каждом слове подписи. Иначе модель начинает орать цветом в каждом месте.

Глава 13

XOR на list/f64

kenga_net.kenga — маленький MLP на списках, без тензорного хоста. Это ступень лестницы: алгоритм сети выражен в языке, а не в вызове `matmul` из `runtime.more` этот файл уже исполняет.

Зачем XOR, если есть `decoder`. Потому что XOR проверяет, что градиент и нелинейность живут в диалекте. `Decoder` проверяет, что тот же диалект тянет внимание и голову. Путать эти проверки — как считать телепортацию доказанной, потому что SVD сошёлся.

ЧАСТЬ V

Своя модель

Глава 14

Decoder, не скачанный граф

Половина Grok — не другой алгоритм. Это тот же decoder: внимание, residual, FFN, норма, LM-head — с другими числами. Кенга уже выражает эту машину. Числа пока игрушечные.

Файл	Что делает
kenga_lm.kenga	decoder на закрытом словаре
kenga_charlm.kenga	тот же decoder, корпус = .kenga
kenga_trigram.kenga	триграмма на list/i64
kenga_birth.kenga	suffix-LM пишет программу
kenga_mm_lm.kenga	PPM+WAV → подпись (linear)
kenga_mm_words.kenga	цвет = один токен + фраза

Актуальные размеры word-decoder в kenga_dec.kenga: D=16, L=1, FF=24, CTX=12. Словарь — двенадцать токенов: девять слов подписи плюс zhivet, v, yazyke. Веса: minds/kenga_mm_we.kt и соседи, в git не кладём. Образец текста: examples/ml/kenga_mm_words_sample.txt.

Промпт языковой строки: ["kenga", "zhivet"], два шага, assert на yazyke. Строку языка крутим втрое чаще за эпоху, иначе её топик «kenga vidit».

Глава 15

Подписи, которые модель умеет

Рабочие фразы сейчас такие:

корпус подписей

```
kenga vidit krasnyj kadr i slyshit ton  
kenga vidit zelenyj kadr i slyshit ton  
kenga vidit sinij kadr i slyshit ton  
kenga zhivet v yazyke
```

Визуальные подписи идут через `generate_words` с `vis-bias` на первом токене. Текстовая строка — `generate_text` без `vis`. Стебель цвета в `kenga_mm_gen.kenga: kra / ze / si`.

F6. Модель на двенадцати токенах может закончить «`kenga zhivet v yazyke`». Это факт про замкнутый словарь, не про понимание языка.

Глава 16

Честный масштаб

Что доказано: язык описывает и гоняет архитектуру большого LM без Python и без PyTorch. Обучение next-token есть. Корпус крошечный.

Нужно для «половины Grok»	Статус
Decoder / attn / FFN в .kenga	есть
Обучение next-token	есть, крошечный корпус
Словарь 50k-128k, контекст 4k-32k	нет
$L \approx 32-64$, $D \approx 4096$, MoE	нет
Триллионы токенов	нет данных
GPU-ядра	нет
Чужие веса GGUF как доказательство	намеренно нет

GTX 1660 SUPER в машине автора часто занята под завязку (~5800/6144 МиБ). Растить D и L на CPU в этой ситуации — трата вечера. Когда карта свободна, меняются fn D() и fn L() в kenga_dec.kenga и корпус. Не раньше.

О1. Называть текущий decoder половиной Grok — ложь. О2. Класть игрушечные веса в Kenga-ai/kenga-mm — ложь. О3. Считать VSIX доказательством языка — путаница слоя.

«Без GPU и корпуса «половина меня» не появится, какой бы синтаксис ни был. Появится, когда этот же decoder жрёт реальные веса и железо.»

ЧАСТЬ VI

Практикум

Глава 17

Задачи

Ответы не подглядывай в `examples/` сразу. Проверка: `kenga run --lite`. Нумерация совпадает с `docs/EXERCISES.md`.

E1. Арифметика

Сумма чисел от 1 до 100, `println`, `return 0`. Ожидай 5050. Файл: `examples/exercises/e01_sum.kenga`.

E2. Список

`[3, 1, 4, 1, 5]` — максимум циклом. Шпаргалка после попытки: `lists_lite.kenga`.

E3. Struct

`Vec2 { x, y }`, `len2 = x2+y2`, точка (3,4) → 25. Бонус: прогони через `lower_c`.

E4. События

Мини-агент `on "tick"`, `emit`, `pump`. Цепочка как в `agent.kenga`.

E5. Тензор

Единичная 2×2 на вектор `[3,4]`. Ожидай `[3,4]`.

E6. SGD

С нулевого `W` выучи `[1,0] → [2]` одним рядом `t_sgd_step`. Шпаргалка: `train_sgd.kenga`.

E7. Картинка

`load_ppm(examples/ml/assets/dot.ppm) → t_mean`. Три числа около 0.5.

E8. Fusion

Сложи `image embedding` с `t_from([3],[0.1,0.1,0.1])`, получи скаляр через `t_matmul` с `[1,1,1]`.

E9. Birth

scripts\kenga-birth.cmd. Модель пишет kenga_born.kenga, запуск печатает 24. Прочитай kenga_birth.kenga — это suffix-LM.

E10-E11. Видишь и слышишь

kenga_mm_lm.kenga — три подписи про цвет и тон.
kenga_mm_gen.kenga — стебель kra/ze/si. kenga_mm_words.kenga — цвет словом и строка kenga zhivet v yazyke.

Глава 18

Лабораторный журнал

Короткие записи, не легенда. Они объясняют, почему книга говорит именно это, а не «язык стремится к независимости».

Запись	Что случилось
host = includes	kenga_lite.c сжат до CRT + rt_*.inc.c
52 selftest	build.cmd не зелёный — хост не готов
XOR на more	kenga_net.kenga исполняется Kenga-VM
экраны строк	без \" c_seed висел на сотнях МБ
birth на more	писал born и сразу печатал 24
12 токенов	zhivet не тонет, если крутить x3
vis-bias	учить whead только на токене цвета
1660 занята	D/L не растим на CPU впустую

Реестр фактов этой книги: F1 runtime без Rust; F2 маркетплейс не догоняет язык; F3 C/Rust — подмости; F4 more → 24; F5 birth не GGUF; F6 двенадцать токенов умеют одну фразу. Три опровержения: O1 половина Grok; O2 заливка игрушки как большой модели; O3 VSIX как доказательство.

Глава 19

Куда смотреть в репозитории

Учить за вечер — docs/LEARN.md. Справочник — docs/LANGUAGE.md. Своя LM — docs/KENGA_LM.md. Hugging Face — docs/HUGGINGFACE.md, и не заливай туда игрушку. Расширение — docs/MARKETPLACE.md, и не поднимай версию без правки редактора.

Сборка книги, которую ты читаешь:

пересборка PDF и EPUB

```
python book\make_cover.py  
python book\make_book.py
```

Исходник текста — book/make_book.py. Если факт в книге расходится с репозиторием, правь книгу или код, не замазывай абзацем про «эволюцию ландшафта».

Послесловие

Первое издание закрывает язык на ступени 3.13: lite-хост из Kenga, компилятор more, петля birth, игрушечный decoder. Следующее издание имеет смысл, когда изменятся числа decoder'a или исчезнет C-хост — не когда накопится ещё одна страница документации.

Что доказывает язык,
если модель на нём
пишет программу
и запускает её?

Первое издание · 2026